SPEECH RECOGNITION · LECTURE 2 / 6

Decoding with a language model

Viterbi on a graph of word models: where the language model enters, how much it should weigh, and why the graph explodes with n.

Data Science Project · Master ISD · Université Paris-Saclay · 10 Nov

This afternoon

Plan

1. From isolated words to connected speech
2. Viterbi: the best state path
3. The word loop: one HMM made of all word models
4. The LM weight λ and the insertion term β
5. Search: exact, greedy and beam

From L1: $\boldsymbol{x}_t$, word HMMs θ_w, forward algorithm, add-k. Notation: last slides.

You implement this afternoon

TODO `train_bigram` slide 12

TODO `WordLoopDecoder._build` slides 7–11

TODO `WordLoopDecoder.decode_emissions` 5, 14, 15

Provided

`WordLoopDecoder.emissions` (the $T \times |\mathcal{S}|$ matrix of $\log b_s(\boldsymbol{x}_t)$), `_lm(u, v)` (slide 11), `prune` (beam search, slide 23), the sweeps `--part connected` and `--part beam`.

Part 1

From isolated words to connected speech

Connected speech: we no longer know where words start

This morning	one utterance = one word: score each word model (forward), keep the best.
Now	one utterance = unknown number of words, unknown boundaries, pauses anywhere.

Idea. Glue all word models into one big HMM (the *word loop*) and find its single best state path. The path gives both the words and their boundaries.

Decision rule used in practice

$$\hat{W} = \arg\max_{W} \Big[\log p(X \mid W) \; + \; \lambda \, \log P(W) \; + \; \beta \, |W| \Big],$$

$$\log p(X \mid W) \approx \max_{q} \; \log p(X, q \mid W) \qquad \text{(Viterbi approximation)}.$$

- $\lambda > 0$: LM weight; β: added once per word ($\beta < 0$ penalises insertions). Not in Bayes' rule (part 4).
- The sum over paths is replaced by its largest term: we search jointly for words and path.
- W is a sequence over $\mathcal{V}_{\text{sil}}$: *silences are words too* and pay $\lambda \log P$ and β. Output $\hat{W}$: that sequence without sil.

The data for connected digits

Utterances (built by `build_utterance`)

- A digit sequence of 3–7 digits.
- For each digit, a random FSDD *test* recording of that digit.
- Joined by -40 dB noise gaps: 150 ms first, then 50–250 ms after each digit; optional white noise at 20/15/10 dB.

Text: a digit Markov chain

Each digit (and the sentence start) has two likely successors that share 80 % of the mass; the 8 others share 20 %.

- LM text: 5 000 sequences sampled from this chain.
- **Matched** test: sequences from the same chain.
- **Mismatched** test: uniformly random digits.

Word models: this morning's, plus a silence model `sil`; vocabulary of the decoder $\mathcal{V}_{\text{sil}} = \mathcal{V} \cup \{\text{sil}\}$, 11 words.

`--part connected` loads `gmm_standard.pkl`, the $S{=}5$, $M{=}2$ model saved by `--part isolated` (trained on the fly if that file is missing).

Part 2

Viterbi: the best state path

Viterbi: dynamic programming over states and time

Best log score of a path that ends in state j at frame t:

$$\delta_t(j) = \max_{q_1,\dots,q_{t-1}} \log p(\boldsymbol{x}_1, \dots, \boldsymbol{x}_t,\ q_1, \dots, q_{t-1},\ q_t = j \mid \theta).$$

Recursion

$$\delta_1(j) = \log \pi_j + \log b_j(\boldsymbol{x}_1)$$

$$\delta_t(j) = \max_i \left[\delta_{t-1}(i) + \log a_{ij}\right] + \log b_j(\boldsymbol{x}_t)$$

$$\psi_t(j) = \arg\max_i \left[\delta_{t-1}(i) + \log a_{ij}\right]$$

Termination and backtrace

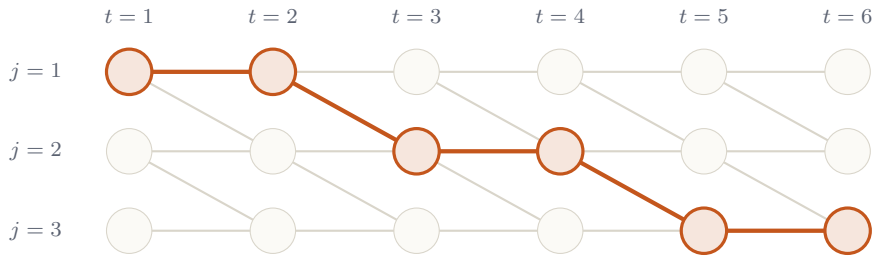
$$\hat{q}_T = \arg\max_j \ \delta_T(j)$$

$$\hat{q}_{t-1} = \psi_t(\hat{q}_t), \quad t = T, \dots, 2$$

$\arg\max$ over the allowed end states: $j = S$ for one word; the word-final states $\mathcal{F}$ for the word loop (part 3).

- Forward recursion with $\max$ for $\operatorname{logsumexp}$; $\psi_t(j)$ remembers *which* predecessor won. $O(TS^2)$ time, $O(TS)$ memory; logs are mandatory (3 s = 300 products).

The trellis: every path is a segmentation, Viterbi keeps the best



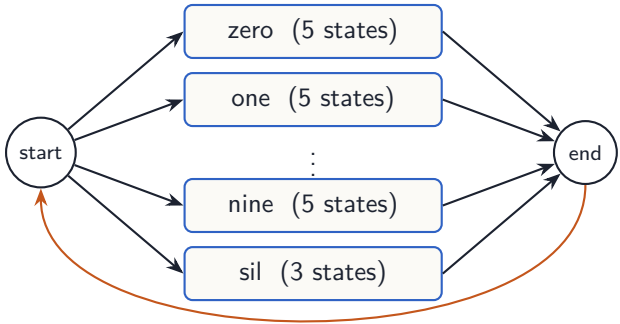
Left-to-right model: arcs only stay (horizontal) or move down one state. A node is a pair (frame t, state j) and holds $\delta_t(j)$.

The highlighted path is a segmentation: frames 1–2 in state 1, 3–4 in state 2, 5–6 in state 3. Its score is $\delta_6(3)$ if it is the best one.

Part 3

The word loop: one HMM made of all word models

The word loop: all word HMMs, connected end to start



- "start" and "end" are drawn for readability only: in the matrix, the last state of u connects *directly* to the first state of every v (next slide).
- The source state already identifies the previous word u: a bigram needs no extra states.

Indexing the states of the word loop

Words $w \in \mathcal{V}_{\text{sil}}$ in a fixed order; word w has S_w states ($S_w = S$ for digits, 3 for sil). Stack them:

$$\mathcal{S} = \{1, \ldots, |\mathcal{S}|\}, \qquad |\mathcal{S}| = \sum_w S_w \ (= 10{\times}5{+}3 = 53), \qquad s = \text{off}(w){+}j \ \text{ for state } j \text{ of word } w,$$

with the offset $\text{off}(w)$ = number of states of the words before w (`self.offsets[w]`). In the code, `self.S` is $|\mathcal{S}|$, not S; S_w is `self.sizes[w]`.

Bookkeeping (provided in `__init__`)

- $\text{first}(w) = \text{off}(w) + 1$,
  $\text{last}(w) = \text{off}(w) + S_w$
- $\text{word}(s)$: `self.word_of_state[s]`
- $\mathcal{F} = \{\text{last}(w) : w \in \mathcal{V}_{\text{sil}}\}$

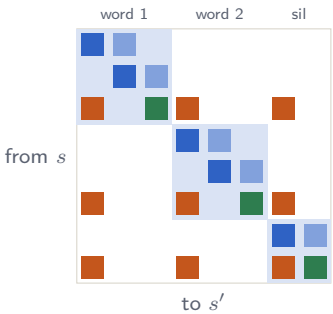
Emissions (provided: `emissions`)

$B_{t,s} = \log b_s(\boldsymbol{x}_t)$, the GMM of state s, $[T \times |\mathcal{S}|]$.

Computed once per utterance; the LM sweep then reuses it (`decode_emissions(B)`).

Code indices start at 0: $\text{first}(w)$ is `offsets[w]`, $\text{last}(w)$ is `offsets[w] + sizes[w] - 1`.

The word-loop transition matrix, block by block



$\log \tilde{a}_{ss'}$ for every pair of states (`self.logA`, $[|\mathcal{S}| \times |\mathcal{S}|]$):

- the block of word w, copied from its own HMM:
- stay: $\log a_{jj}^{(w)}$ move: $\log a_{j,j+1}^{(w)}$
- final self-loop: $\log \varphi$
- entry, $\text{last}(u) \to \text{first}(v)$, all u, v (including $u = v$):

$$\log(1 - \varphi) + \lambda \log P(v \mid u) + \beta$$

- everything else (white): $-\infty$

Mark the entry pairs: $\mathcal{E} = \{(\text{last}(u), \text{first}(v))\}$ (`self.is_entry`), needed to cut the path into words.

Leaving a word, starting and ending the utterance

Why φ: the final state must be able to leave

In the trained word model the last state is absorbing: $a_{SS}^{(w)} = 1$ (isolated words end there). In the loop we replace it:

- stay with probability φ (code: `exit_logp` $= \log \varphi$, default $\log 0.5$),
- leave with probability $1 - \varphi$, shared among the next words v by the LM.

Start: the first word pays the LM too

$$\log \tilde{\pi}_{\text{first}(v)} = \lambda \log P(v \mid \langle \text{s} \rangle) + \beta,$$

$\log \tilde{\pi}_s = -\infty$ for every other state (`self.start`).

End

The path must end in a word-final state: $\hat{q}_T = \arg\max_{s \in \mathcal{F}} \delta_T(s)$. Simplification: no $P(\langle/\text{s}\rangle \mid u)$ term.

With $\lambda \neq 1$ and $\beta \neq 0$ the rows of $\tilde{a}$ no longer sum to one: δ is a *score*, not a log-probability. That is fine for an $\arg\max$.

`_build` in the code

TODO `WordLoopDecoder._build(self)`: set three attributes

- `self.logA` $[|\mathcal{S}| \times |\mathcal{S}|]$, float, filled with `-np.inf`: $\log \tilde{a}$.
- `self.start` $[|\mathcal{S}|]$, float, `-np.inf` except at $\text{first}(v)$: $\log \tilde{\pi}$.
- `self.is_entry` $[|\mathcal{S}| \times |\mathcal{S}|]$, bool: `True` on $\mathcal{E}$.

Where the pieces are

- $a_{ij}^{(w)}$: `self.wm.models[w].transmat_` (`hmmlearn`). Take `np.log` under `np.errstate(divide="ignore")` so that zeros become $-\infty$ silently.
- $\log \varphi$: `self.exit_logp`; $\log(1-\varphi)$: `np.log1p(-np.exp(self.exit_logp))`.

Provided: `self._lm(u, v)`

Returns $\lambda \log \max\left(P(v \mid u), 10^{-12}\right) + \beta$, read from `self.bigram[u][v]`.

It already contains λ and β: do not add β again.

Entry: `logA[last(u), first(v)]` $= \log(1-\varphi) +$ `self._lm(u, v)`.
Start: `start[first(v)]` $=$ `self._lm("<s>", v)`.

Keys of `self.bigram` are the literal strings: `"<s>"`, the digit words, `"sil"`.

The bigram LM, with a silence word

TODO `train_bigram(sentences, vocab, k=0.1, sil_token="sil", p_sil=0.15)`

1. Add-k bigram over the digits (each sentence preceded by $\langle \mathrm{s} \rangle$; no $\langle /\mathrm{s} \rangle$ here):

$$P_k(v \mid u) = \frac{\#(u\,v) + k}{\sum_{v' \in \mathcal{V}} \#(u\,v') + k\,V}, \qquad u \in \{\langle \mathrm{s} \rangle\} \cup \mathcal{V},\ v \in \mathcal{V}.$$

2. Silence can occur between any two words:

$$P(v \mid u) = (1 - p_{\mathrm{sil}})\, P_k(v \mid u), \qquad P(\mathrm{sil} \mid u) = p_{\mathrm{sil}}.$$

3. After a silence, a unigram built from the same smoothed counts, and a tiny silence self-transition:

$$P(v \mid \mathrm{sil}) = (1 - 10^{-6})\, \frac{\sum_{u \in \{\langle \mathrm{s} \rangle\} \cup \mathcal{V}} \big(\#(u\,v) + k\big)}{\sum_{v' \in \mathcal{V}} \sum_{u \in \{\langle \mathrm{s} \rangle\} \cup \mathcal{V}} \big(\#(u\,v') + k\big)}, \qquad P(\mathrm{sil} \mid \mathrm{sil}) = 10^{-6}.$$

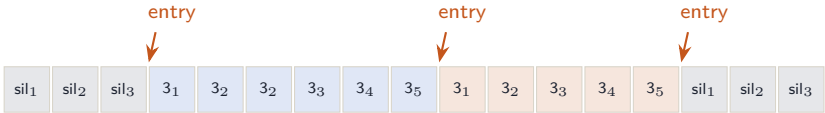
`train_bigram`: output format and notes

- Return `{u: {v: P(v|u)}}` with the literal keys `"<s>"`, the digit words, `"sil"`. Every row, sil included, sums to 1 (tested to 10^{-9}).
- With `sil_token=None`, only step 1.
- Step 3 is an approximation: after a pause the bigram history is forgotten. Keeping it would need one copy of sil per previous word.
- Acoustic-only baseline: `bigram=None` gives the uniform "LM" $P(v \mid u) = 1/|\mathcal{V}_{\mathrm{sil}}|$.

From the best state path to words

After the backtrace we have $\hat{q}_1, \ldots, \hat{q}_T$. A new word starts at frame t iff $(\hat{q}_{t-1}, \hat{q}_t) \in \mathcal{E}$:

$\hat{W} = (\mathrm{word}(\hat{q}_1))$ followed by $\mathrm{word}(\hat{q}_t)$ for every $t \geq 2$ with $(\hat{q}_{t-1}, \hat{q}_t) \in \mathcal{E}$, then drop sil.



frames $t = 1, \ldots, 17$; label $= \mathrm{word}_{\mathrm{state}}$

words: sil, three, three, sil $\rightarrow$ $\hat{W}$ = "three three"

- Detecting a change of word label is not enough: "three three" never changes label. Only the entry transition $\mathrm{last}(3) \rightarrow \mathrm{first}(3)$ reveals the second word.

`decode_emissions`: the whole algorithm

TODO `decode_emissions(B, drop=("sil",))`

In: B $[T \times |\mathcal{S}|]$; uses $\log \tilde{\pi}$, $\log \tilde{a}$, $\mathcal{E}$ from `_build`.

$\delta \leftarrow$ `prune`$(\log \tilde{\pi} + B_{1,:})$ ▷ $[|\mathcal{S}|]$
for $t = 2, \ldots, T$ **do**
  $C \leftarrow \delta[:, \text{None}] + \log \tilde{a}$ ▷ $C_{ss'}$: from s to s'
  $\psi_t \leftarrow \arg\max_s C_{s,:}$; $\delta \leftarrow$ `prune`$(\max_s C_{s,:} + B_{t,:})$
$\hat{q}_T \leftarrow \arg\max_{s \in \mathcal{F}} \delta[s]$ ▷ all $-\infty$: $\arg\max_s \delta[s]$
for $t = T, \ldots, 2$ **do**
  $\hat{q}_{t-1} \leftarrow \psi_t[\hat{q}_t]$
cut into words at entry transitions; drop sil
return list of words

- Each step is one NumPy broadcast: `C = delta[:, None] + logA`, then `argmax(axis=0)`.
- Time $O(T|\mathcal{S}|^2)$: here $53^2 \approx 2\,800$ per frame. A sparse implementation is $O(T(|\mathcal{S}| + |\mathcal{V}_{\text{sil}}|^2))$.
- Memory: ψ is $[T \times |\mathcal{S}|]$ integers.
- `prune` is provided (beam search, slide 23). Without a beam it changes nothing; call it at every frame anyway.
- All-$-\infty$ columns early on (state 3 at $t = 2$) are harmless: δ stays $-\infty$. Check that $\max_{s \in \mathcal{F}} \delta_T(s)$ is finite.

Part 4

The LM weight λ and the insertion term β

Why the language model needs a weight

Scale mismatch

- $\log p(X, q \mid W)$ sums ≈ 100 emission terms per second, each of the order of tens of nats.
- $\log P(W)$ adds one term per *word*, of the order of -1 to -3 nats.
- Frames are not independent (25 ms windows overlap by 15 ms; Δ features span ± 4 frames): the product over frames counts the same evidence several times, so the acoustic score is overconfident.

λ: LM weight (`lm_weight`)

Multiplies every $\log P(v \mid u)$: rebalances prior against evidence.

- Too small: acoustics alone.
- Too large: the LM overrides what was said.

Swept in the lab: $\lambda \in \{0.5, 1, 2, 4, \ldots, 256\}$ (powers of 2).

The insertion term β, and its scale

β: added once per word (`word_penalty`)

Every entry transition (and the start) adds β, so a path through $|W|$ words gets $\beta|W|$, *silences included*.

- $\beta \ll 0$: few words, deletions.
- β near 0: short spurious words, insertions.

Sign convention here: $\beta < 0$ is a penalty. (Day 2 decoders use a "word bonus": same role, opposite sign.)

Scale: tens to hundreds of nats, not 1

A 5-state word lasts ≈ 40 frames, each with a log-density around -40 to -60: word-level differences are tens of nats.

- $\beta = \pm 4$ changed nothing.
- Reference grid:
  $\beta \in \{-600, -400, -300, -200, -150, -100, -60, -30, -15, 0\}$.
- $\beta = 0$ wins only for $\lambda \geq 128$: there the LM already penalises every word.

Tune λ and β on a development set, never on the test set

Protocol (reference run)

- Split the utterances: the first 30 are **dev**, the rest **test**.
- For each λ: grid over β on dev, keep the best β.
- Report WER on test only, once per (λ, β).
- Check the optimum is not on the grid edge (it was, at first).

A confound to know

With the uniform "LM", every entry costs

$$\log(1-\varphi) + \lambda \log \tfrac{1}{|\mathcal{V}_{\text{sil}}|} + \beta$$
$$= \log(1-\varphi) + \beta - \lambda \log |\mathcal{V}_{\text{sil}}|.$$

A large λ *alone* acts as an insertion penalty.

$\Rightarrow$ Compare LMs at a **fixed** λ to isolate what the LM *content* brings.

What the bigram LM bought on connected digits

Word error rate $\mathrm{WER} = (\text{substitutions} + \text{deletions} + \text{insertions})/\text{reference words}$ on the 120 test utterances (150 generated, 30 dev) of `--part connected` with its defaults; GMM-HMM with $S = 5$, $M = 2$; β tuned on dev for each cell.

Condition	Acoustic only	Best λ^*	$\lambda = 64$
Matched LM, clean	4.4 %	1.5 % (λ=32)	2.3 %
Matched LM, 20 dB	26.4 %	9.2 % (λ=32)	11.0 %
Matched LM, 15 dB	45.0 %	18.6 % (λ=64)	18.6 %
Mismatched LM, clean	1.7 %	1.7 % ($\lambda \leq 4$)	7.0 %

- The prior matters most when the acoustic evidence is weak (noise).
- It hurts when it is wrong (mismatched text) *and* too heavy ($\lambda = 64$). Far too heavy hurts even when right: matched, clean, $\lambda = 256$ gives 18.0 %.

* $\lambda \in \{0.5, 1, \ldots, 256\}$ picked as the best of the sweep *on test*: an optimistic number. In your report, pick λ on dev as well.

Four traps we fell into while building this lab

Silence

Untrimmed recordings: the word “six” absorbed every pause. All errors were insertions. Fix: a silence model trained on recording edges.

Penalty scale

$\beta = \pm 4$ changed nothing: the acoustic scores live in tens of nats.

Normalisation

Isolated training words and long utterances had different feature statistics. Fix: one global normaliser (L1).

Grid edge

The best β sat on the edge of the grid: part of the “LM gain” was a penalty gain.

Part 5

Search: exact, greedy and beam

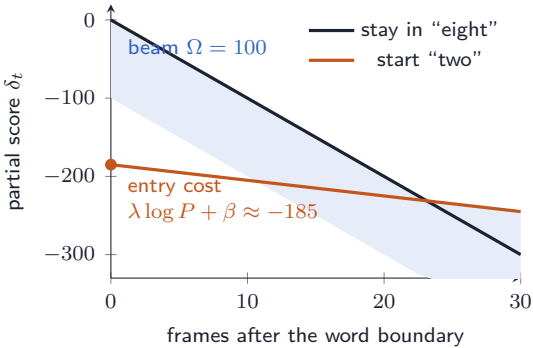
The decoding graph grows with the n-gram order

The entry score $\lambda \log P(v \mid h) + \beta$ must know the history h (the last $n-1$ words). In a bigram, $h = u$ is the word we leave: free. Beyond that, *each word model must be copied once per possible history*.

Order	History to remember	Copies per word	10 digits	10^5 words
unigram	nothing	1	10	10^5
bigram	previous word (the source state)	1	10	10^5
trigram	previous word, inside the graph	V	100	10^{10}
4-gram	two previous words	V^2	1 000	10^{15}

- Columns "10 digits" and "10^5 words": total word-model copies ($V\times$ copies per word); multiply by S for states.
- Large-vocabulary systems compile the graph (weighted finite-state transducers: compose, determinise, minimise) and do not visit all of it: they search it with a *beam* (next slides).

Viterbi is exact search; greedy search is not



Stylised slopes; the entry cost is the real one ($\lambda = 32$, $\beta = -150$).

- **Viterbi keeps, for every state, the best path into it.** Paths that meet in a state have the same future, so the worse one can be dropped *now* (recombination). Nothing else is dropped: the search is **exact**.
- **Greedy** keeps one hypothesis per frame, the locally best, and never revisits it. A new word pays ≈ -185 at once and catches up only later: greedy never leaves the first word (91 % WER on our test set).
- **Beam search** sits in between: keep every hypothesis still within reach of the best (shaded band). A band narrower than the entry cost repeats greedy's mistake.

Beam search: two rules to decide which hypotheses survive

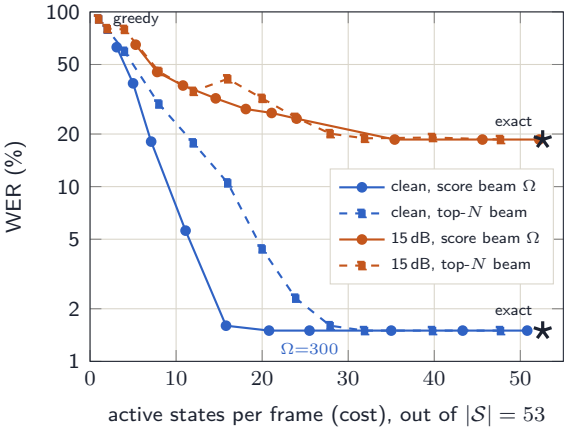
After computing δ_t, before expanding frame $t+1$; pruned states get $\delta_t(s) \leftarrow -\infty$

Score beam Ω: keep s iff $\delta_t(s) \geq \max_{s'} \delta_t(s') - \Omega$
Top-N beam N_{beam}: keep the N_{beam} states with the largest $\delta_t(s)$ (both can be combined)
In the code: `WordLoopDecoder(..., beam=`Ω`, n_beam=`N_{beam}`)`; the provided `self.prune` applies them.

Search	Kept per frame	Cost per frame	Finds the model's best?
Exact Viterbi	all $\lvert\mathcal{S}\rvert$ states	$\lvert\mathcal{S}\rvert\times$ arcs per state	always
Score beam	adapts: few if one hypothesis dominates, many if ambiguous	$\propto$ active states	if Ω is wide enough
Top-N beam	exactly N_{beam}	fixed: predictable latency, memory	if N_{beam} is large enough
Greedy	1 ($N_{\text{beam}} = 1$)	minimal	rarely

- The beam must be wider than the cost of entering a word, or no new word survives. Real decoders spread that cost over the word (LM look-ahead) or give word ends their own beam.
- If the state is the whole history (neural LMs, attention decoders: later days), paths never meet: exact search is impossible, beam search *is* the decoder.

Measured: the beam width is an accuracy–cost knob



`--part beam`: 120 test utterances, bigram at its best λ, β (slide 19).

- **Free savings**: clean, $\Omega = 300$ keeps half the states (25.5) at the same 1.5 % WER; so does $N_{\text{beam}} = 32$.
- **A cliff** below $\Omega \approx 200$: the word-entry cost (≈ 185).
- **The safe width depends on the condition**: at 15 dB, $\Omega = 300$ costs 6 points (18.6 $\rightarrow$ 24.5 %). Tune it on dev, in the hardest condition.
- Greedy: 91 % WER.

Only 53 states here, and dense `numpy` code is not faster when pruned. Large decoders keep thousands of active states out of millions.

Search error or model error? Check before widening the beam

Write $\text{score}(W) = \log p(X \mid W) + \lambda \log P(W) + \beta |W|$, the objective that the decoder maximises (slide 3).

Two different failures

- Search error: the decoder returns $\hat{W}$, but some W has a higher score. The model was right; the search missed it.
- Model error: the model's own best W is wrong. No search can fix this.

The test

Score the reference W^{ref} with the same model (Viterbi restricted to its words: forced alignment). $\text{score}(W^{\text{ref}}) > \text{score}(\hat{W})$: search error $\Rightarrow$ widen the beam. Otherwise: model error $\Rightarrow$ change features, model, LM, λ or β.

Measured (15 dB)

Exact search: 18.6 % WER, all model errors. $\Omega = 350$: 13 % of utterances have a search error, yet the same WER (it does not always change the words). $\Omega = 300$: 36 %, WER 24.5 %.

For you

This afternoon: `--part beam`, then choose a width for your system and justify it in `DECISIONS.md`. Day 2: the same knob (N_{beam}) for CTC decoders, where beam search is the only way to search.

Lab, this afternoon, and delivery d1

Implement (each TODO ↔ a slide)

TODO	`train_bigram`	slide 12
TODO	`WordLoopDecoder._build`	9–11
TODO	`decode_emissions`	5, 14, 15

Check: `pytest -m d1`

Run

```
python -m speech.run_classical_baseline \
  --fsdd <dir> --part connected
```

λ sweep × SNR, matched vs mismatched LM; β tuned on dev only.

Then `--part beam`: WER vs active states (slide 24); pick a beam width.

Deliver d1 (git tag)

- rows: `gmm_isolated` ≥ 8, `ngram_lm` ≥ 8, `gmm_connected` ≥ 6, `gmm_beam` ≥ 4
- figures + at least one decision in `DECISIONS.md`

Stretch

A trigram decoder: V copies of each word (slide 21). Worth an oral question, not required.

Try your decoder live

Same demo (`python -m asr_demo.server`); say a **digit string**, with short pauses.

Menu entry	What it is	Works once
`digits/3-word-loop`	word loop + Viterbi, uniform LM, $\beta = -300$	`_build`, `decode_emissions`
`digits/4-word-loop+bigram`	the same + bigram, $\lambda = 2$, $\beta = -200$	... and `train_bigram`

- The bigram is trained on strings of a digit grammar (each digit has two likely successors). "New prompt" draws from that grammar: there the LM should help. Then say random digits.
- Record once, then "Replay" the recording on the other entry.
- Your own λ, β: register another entry in `my_backends.py` (example at the top of `asr_demo/digits.py`).

Notation (1/3): evaluation, signal and features

Comparing systems

$\mathcal{M},\ \mathcal{M}'$	systems (configurations)
$f_o,\ o \in \mathcal{O}$	objectives, all "lower is better"
$\mathcal{M} \succ \mathcal{M}'$	$\mathcal{M}$ dominates $\mathcal{M}'$
$c_{\max}$	budget on a cost (e.g. latency)
N_{sys}	number of systems compared
$\hat{p},\ N_{\text{test}},\ z$	measured accuracy, test files, Gaussian quantile (1.96)
R	number of timed requests (p50, p90 over them)
RTF	compute time / audio duration
WER	(sub. + del. + ins.) / reference words

Degradations

$\epsilon,\ g$	noise signal, its gain
SNR	signal-to-noise ratio (dB)
rir, T_{60}	room impulse response, reverberation time
$F,\ \mu_{\text{law}}$	μ-law compander, $\mu_{\text{law}} = 255$

Signal and features

$y[\tau]$	waveform, sample index τ
f_{s}	sampling rate (8 kHz on FSDD)
$N_w,\ N_h$	window (200) and hop (80) in samples
$\text{win}[\tau]$	Hann window
$t,\ T$	frame index $1, \dots, T$ (10 ms hop)
f	FFT bin; $P_t[f]$ power spectrum
$\nu,\ N_{\text{mel}}$	mel band, number of bands (40)
$H_\nu[f]$	mel filter ν
$E_t[\nu],\ \tilde{E}_t[\nu]$	mel energy, its log (dB)
$\ell,\ \rho_\ell$	cepstral index, DCT scale
$c_t[\ell]$	MFCC ℓ of frame t, $\ell = 0, \dots, 12$
$\Delta c_t,\ \Delta^2 c_t$	1st and 2nd time derivatives (± 4 frames)
$\boldsymbol{x}_t \in \mathbb{R}^D$	feature vector of frame t, $D = 39$
x_d	component d of a vector $\boldsymbol{x}$
X	observation $(\boldsymbol{x}_1, \dots, \boldsymbol{x}_T)$
$\hat{\boldsymbol{\mu}},\ \hat{\boldsymbol{\sigma}}$	global normalisation statistics

Notation (2/3): acoustic models

HMM of one word w

θ_w	parameters $(\boldsymbol{\pi}, A, \{b_j\})$ of word w
$i,\ j$	states, $1, \dots, S$ (S per word)
q_t	hidden state at frame t
π_j	$P(q_1 = j)$, initial probability
a_{ij}	$P(q_t = j \mid q_{t-1} = i)$, transition
κ	time spent in one state (frames)
$b_j(\boldsymbol{x})$	$p(\boldsymbol{x}_t = \boldsymbol{x} \mid q_t = j)$, emission density
$m,\ M$	Gaussian component, $1, \dots, M$
$\omega_{jm}, \boldsymbol{\mu}_{jm}, \boldsymbol{\sigma}^2_{jm}$	weight, mean, diagonal variance
$\xi_m,\ \mathrm{logsumexp}$	arguments of a log-sum-exp

Algorithms

$\alpha_t(j)$	forward variable (sum over paths)
$\delta_t(j)$	Viterbi score (max over paths)
$\psi_t(j)$	back-pointer: best predecessor
$\hat{q}_t$	state of the best path
$\zeta_t(j),\ \zeta_t(i,j)$	EM posteriors $P(q_t{=}j \mid X)$, $P(q_{t-1}{=}i, q_t{=}j \mid X)$
$\zeta_t(m)$	responsibility of Gaussian m for frame t
$\theta^{\mathrm{old}},\ \theta^{\mathrm{new}}$	parameters before / after one EM iteration
Ω	score beam: keep δ within Ω of the best
N_{beam}	top-N beam: number of hypotheses kept

Word loop (L2)

$s \in \mathcal{S}$	a state of the loop; $\lvert\mathcal{S}\rvert$ (`self.S`)
$S_w,\ \mathrm{off}(w)$	states of word w, its offset
$\mathrm{first}(w),\ \mathrm{last}(w)$	entry and final state of word w
$\mathrm{word}(s)$	word that state s belongs to
$\mathcal{F}$	word-final states $\{\mathrm{last}(w)\}$
$\mathcal{E}$	entry transitions $\mathrm{last}(v) \to \mathrm{first}(w)$

Notation (3/3): words, language models, decoding

Words

$\mathcal{V},\ V$	vocabulary (10 digits), $V = \lvert\mathcal{V}\rvert$
$u,\ v,\ w$	words
W	word sequence $(w_1, \ldots, w_L)$, $\lvert W\rvert = L$
$\hat{W}$	recognised word sequence
$\mathcal{V}_{\text{sil}}$	$\mathcal{V} \cup \{\text{sil}\}$, with a silence word

Decoding

λ	LM weight (`lm_weight`)
β	word insertion term (`word_penalty`), $\beta < 0$
p_{sil}	probability of a silence between words
$\text{score}(W)$	$\log p(X \mid W) + \lambda \log P(W) + \beta \lvert W\rvert$
W^{ref}	reference transcription

n-gram language models

n	order: $n-1$ words of history
$h,\ h_i$	a history; the history of position i
$h^{(r)}$	last $r-1$ tokens of h
$\langle\text{s}\rangle,\ \langle/\text{s}\rangle$	start / end-of-sentence tokens
$\mathcal{V}'$	$\mathcal{V} \cup \{\langle/\text{s}\rangle\}$: what the LM can predict
$\#(\cdot)$	count in the training text
$P_{\text{ML}},\ P_k,\ P_{\text{int}}$	max. likelihood, add-k, interpolated
k	add-k smoothing constant
η_r	interpolation weight of order r
$P_{\text{KN}},\ P_{\text{cont}},\ \gamma$	Kneser–Ney, continuation prob., discount
$\mathcal{D},\ L_W$	test sentences; length of sentence W
K	number of predicted tokens in $\mathcal{D}$
PP	perplexity

All logarithms are natural (scores in nats). λ always denotes the LM weight; HMM parameters are θ.